

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf\_27df53c8-8e0\agent-ae30127464035bec8.jsonl`
- SHA-256: `17fb762844e985ac6ba897db5d5ce78e8b106bd1d856762b86ed35f7154987b3`
- Source modified: `2026-06-10T16:31:14+00:00`
- Imported at: `2026-07-05T16:48:21+00:00`
- project: `wf\_27df53c8-8e0`
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`

Transcript

1. user (2026-06-10T16:28:23.461Z)

You are an adversarial verifier. A code reviewer audited the repo at C:/Users/avidu/Projects/muneem and reported this finding:

TITLE: HDFC parser persists all rows even when the parse is UNRECONCILED, and `txns list` never filters by document status
SEVERITY CLAIMED: high
FILE: src/muneem/parsers/hdfc.py (line 435-458)
DESCRIPTION: Unlike AxisParser/AUParser (which store nothing on an unverified parse), HDFCParser.parse_and_insert inserts every transaction into hdfc_transactions BEFORE computing the verdict and never removes them when the verdict is 'unreconciled' ? the status is recorded but the suspect rows stay. The only read path, `muneem txns list hdfc` (cli.py:347-361 via db.list_hdfc_transactions, db.py:437-440), does not join documents or filter/annotate by status, so rows from a known-bad parse (e.g. a page whose 'Narration'/'Closing' header line failed to extract ? _table_body returns [] for that page and silently drops the whole page's transactions) are served indistinguishably from trusted rows. Related: _parse_transaction line 219 fabricates `closing\_balance=0.0` when the balance token fails _AMOUNT_RE, persisting an invented zero balance on such rows.
EVIDENCE QUOTED: hdfc.py:435-447: `for t in stmt.transactions:
dbmod.insert_hdfc_transaction(...)` runs unconditionally; :454 `verdict = Confidence.RECONCILED
if ok else Confidence.UNRECONCILED` only sets status. db.py:437-440: `SELECT * FROM
hdfc_transactions ORDER BY id DESC LIMIT ?` ? no status filter. hdfc.py:219:
`closing\_balance=closing if closing is not None else 0.0`.
SUGGESTED FIX: Apply the same gate as Axis/AU: compute the verdict first (parse + summary are both pure reads) and insert rows only when reconciled ? or at minimum exclude/flag rows whose document status != 'reconciled' in list queries. Replace the fabricated 0.0 closing balance with None or a parse rejection.

Your job is to try to REFUTE it. Your lens: correctness: trace the actual code path the finding describes and check the claimed behavior is what the code really does (read the file and its callers/tests).

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").

- Do not use WebSearch/WebFetch and do not send any repo content to any external service.
Structured output only.

2. user (2026-06-10T16:28:28.248Z)

```
1      """HDFC savings / transaction-account statement parser.
2
3      **Approach:** coordinate / column clustering on **pdfplumber (MIT)** ? not the
4      default ``extract_tables()`` and not a ready-made library. HDFC's transaction
5      table is *semi-ruled*, where ``extract_tables()`` is unreliable; the deep-research
6      decision (see the ``hdfc-parser-tech-decision`` memory) is to cluster words by
7      position. pdfplumber's whole chain is permissive and it opens password-protected
8      statements directly.
9
10     **Layout.** Each transaction is one *primary* line ? the transaction date at the
11     far left (``x0?52``) plus three right-aligned amount columns (Withdrawal, Deposit,
12     Closing Balance) ? followed by zero or more *continuation* lines that wrap the
13     narration and carry the value date (``dd/mm/yyyy`` at ``x0>110``) and the
14     cheque/reference number. We group words into visual lines by ``top``, detect
15     primary lines by a leading date + a closing-balance amount, attach continuations,
16     and read columns by x-position.
17
18     **Correctness is checked, not assumed.** ``reconcile`` verifies the running
19     balance (closing[i] == closing[i-1] - withdrawal + deposit) and
20     ``validate_against_summary`` cross-checks the parse against HDFC's own end-of-
21     statement SUMMARY (Dr/Cr counts + opening/closing balance). Together they prove a
22     parse complete without inspecting any individual value ? which is how we validate
23     against real (sensitive) statements. No statement content is logged here.
24     """
25
26     from __future__ import annotations
27
28     import csv
29     import re
30     from dataclasses import dataclass, field
31     from pathlib import Path
32
33     import pdfplumber
34
35     from muneem import __version__
36     from muneem import db as dbmod
37     from muneem.errors import ParseError
38     from muneem.redaction import redact_filename
39
40     from . import register_parser
41     from .base import StatementParser
42     from .concepts import ConceptRow, StatementConcepts
43     from .profiles import HDFC_PROFILE
44     from .validation import (
45         BookendReport,
46         Confidence,
47         ReconcileReport,
48         reconcile_running_balance,
49         verify,
50     )
51
52     # --- Column geometry (PDF points; A4-portrait HDFC detailed statement) -----
53     # Left-aligned columns key on x0; right-aligned amount columns key on their right
54     # edge (x1), which is stable regardless of the number's magnitude.
55     _DATE_MAX_X0 = 70.0 # transaction date sits at x0?52; the value date is at x0>110
56     _NARR_MIN_X0 = 100.0 # narration band starts at x0?111
57     _AMOUNT_MIN_X0 = 295.0 # amount columns start at x0?306; narration never crosses ?290
58     _WITHDRAWAL_MAX_X1 = 400.0 # withdrawal right edge ?358
59     _DEPOSIT_MAX_X1 = 490.0 # deposit right edge ?443; closing balance right edge ?561
60     _LINE_TOL = 2.5 # words within this many points of each other share a visual line
```

```

61     _CONT_MAX_SPAN = 70.0 # a continuation line is at most this far below its primary line
62
63     _DATE_RE = re.compile(r"^d{2}/d{2}/d{4}$")
64     # Indian-format money: optional minus, comma groups, exactly two decimals, optional
65     Cr/Dr.
66     _AMOUNT_RE = re.compile(r"^-[?[\d,]+\.\d{2}(?:cr|dr)?$", re.IGNORECASE)
67     _FOOTER_MARKERS = {"STATEMENT", "SUMMARY"}
68
69     @dataclass
70     class Transaction:
71         date: str # dd/mm/yyyy, as printed
72         narration: str
73         withdrawal: float
74         deposit: float
75         closing_balance: float
76         value_date: str | None = None
77         ref: str | None = None
78         page: int = 0
79
80         def to_concept(self) -> ConceptRow:
81             """Map HDFC's native row onto the bank-agnostic concept vocabulary (the
82 verifier's input).
83
84             Withdrawal ? ``debit``, deposit ? ``credit``, closing balance ? ``balance``. The
85             ``page``
86             field is HDFC-internal provenance and has no concept equivalent.
87             """
88             return ConceptRow(
89                 txn_date=self.date,
90                 narration=self.narration,
91                 debit=self.withdrawal,
92                 credit=self.deposit,
93                 balance=self.closing_balance,
94                 value_date=self.value_date,
95                 reference=self.ref,
96             )
97
98     @dataclass
99     class Statement:
100         path: Path
101         transactions: list[Transaction] = field(default_factory=list)
102         account_number: str | None = None
103
104     # HDFC's SUMMARY block and its cross-check are now expressed in the shared concept
105     vocabulary +
106     # verifier (the concept-core refactor). These back-compat names keep HDFC's public
107     surface stable:
108     # StatementSummary == the statement-level concepts a SUMMARY block supplies
109     # ValidationReport == the verifier's bookend report (per-row walk + printed-figure
110     checks)
111     StatementSummary = StatementConcepts
112     ValidationReport = BookendReport
113
114     def _to_amount(token: str) -> float | None:
115         """Parse an HDFC money token to a float, or None if it is not money.
116
117         Unlike base.safe_amount we keep ``0.00`` as ``0.0`` (not None): a zero amount
118         is real information and the reconciliation math needs it.
119         """
120         if not _AMOUNT_RE.match(token):
121             return None

```

```

120         cleaned = re.sub(r"(?i)(cr|dr)$", "", token).replace(",", "")
121     try:
122         return float(cleaned)
123     except ValueError:
124         return None
125
126
127 def _group_lines(words: list[dict]) -> list[list[dict]]:
128     """Group words into visual lines by ``top`` (tolerant of sub-pixel jitter)."""
129     lines: list[list[dict]] = []
130     anchor_top: float | None = None
131     for w in sorted(words, key=lambda w: (w["top"], w["x0"])):
132         if lines and anchor_top is not None and abs(w["top"] - anchor_top) <= _LINE_TOL:
133             lines[-1].append(w)
134         else:
135             lines.append([w])
136             anchor_top = w["top"]
137     return [sorted(line, key=lambda w: w["x0"]) for line in lines]
138
139
140 def _line_top(line: list[dict]) -> float:
141     return min(w["top"] for w in line)
142
143
144 def _has_narration(line: list[dict]) -> bool:
145     return any(_NARR_MIN_X0 <= w["x0"] < _AMOUNT_MIN_X0 for w in line)
146
147
148 def _has_amount(line: list[dict]) -> bool:
149     return any(w["x0"] >= _AMOUNT_MIN_X0 and _AMOUNT_RE.match(w["text"]) for w in line)
150
151
152 def _has_leading_date(line: list[dict]) -> bool:
153     first = line[0]
154     return bool(_DATE_RE.match(first["text"]) and first["x0"] < _DATE_MAX_X0)
155
156
157 def _has_footer_marker(line: list[dict]) -> bool:
158     return any(w["text"].upper() in _FOOTER_MARKERS for w in line)
159
160
161 def _is_primary(line: list[dict]) -> bool:
162     """A transaction's first line: a date at the far left + a closing-balance amount.
163
164     Requiring the closing-balance amount guards against stray dates in the page
165     header / account-summary box (e.g. the statement period).
166     """
167     if not _has_leading_date(line):
168         return False
169     return any(
170         w["x0"] >= _AMOUNT_MIN_X0 and w["x1"] >= _DEPOSIT_MAX_X1 and
171         _AMOUNT_RE.match(w["text"])
172         for w in line
173     )
174
175 def _is_continuation(line: list[dict]) -> bool:
176     """A narration-wrap line: narration-band text, no leading date, no amounts."""
177     return _has_narration(line) and not _has_leading_date(line) and not
178     _has_amount(line)
179
180 def _parse_transaction(
181     primary: list[dict], continuations: list[list[dict]], page_no: int
182 ) -> Transaction:

```

```

183 withdrawal = deposit = 0.0
184 closing: float | None = None
185 narr: list[tuple[float, float, str]] = [] # (top, x0, text) for ordered reassembly
186
187 for line in (primary, *continuations):
188     for w in line:
189         x0, x1, text = w["x0"], w["x1"], w["text"]
190         if x0 >= _AMOUNT_MIN_X0:
191             val = _to_amount(text)
192             if val is None:
193                 continue
194             if x1 < _WITHDRAWAL_MAX_X1:
195                 withdrawal = val
196             elif x1 < _DEPOSIT_MAX_X1:
197                 deposit = val
198             else:
199                 closing = val
200         elif x0 >= _NARR_MIN_X0:
201             narr.append((w["top"], x0, text))
202
203 value_date: str | None = None
204 parts: list[str] = []
205 for _top, _x0, text in sorted(narr):
206     if value_date is None and _DATE_RE.match(text):
207         value_date = text # the value date, lifted out of the narration band
208         continue
209     parts.append(text)
210
211 ref_candidates = [p for p in parts if p.isdigit() and len(p) >= 9]
212 ref = max(ref_candidates, key=len) if ref_candidates else None
213
214 return Transaction(
215     date=primary[0]["text"],
216     narration=" ".join(parts),
217     withdrawal=withdrawal,
218     deposit=deposit,
219     closing_balance=closing if closing is not None else 0.0,
220     value_date=value_date,
221     ref=ref,
222     page=page_no,
223 )
224
225
226 def extract_transactions_from_lines(lines: list[list[dict]], page_no: int = 0) ->
list[Transaction]:
227     """Pure, PDF-free core: turn grouped table-body lines into transactions.
228
229     Kept separate from PDF I/O so the column logic is unit-testable on synthetic
230     word lists, with no real data.
231     """
232     primaries = [i for i, line in enumerate(lines) if _is_primary(line)]
233     transactions: list[Transaction] = []
234     for k, idx in enumerate(primaries):
235         stop = primaries[k + 1] if k + 1 < len(primaries) else len(lines)
236         primary = lines[idx]
237         primary_top = _line_top(primary)
238         continuations: list[list[dict]] = []
239         for m in range(idx + 1, stop):
240             line = lines[m]
241             if _has_footer_marker(line) or _line_top(line) - primary_top >
_CONT_MAX_SPAN:
242                 break
243             if _is_continuation(line):
244                 continuations.append(line)
245             transactions.append(_parse_transaction(primary, continuations, page_no))

```

```

246         return transactions
247
248
249     def _table_body(lines: list[list[dict]]) -> list[list[dict]]:
250         """Lines below the column header (drops the per-page account-summary box)."""
251         header_top: float | None = None
252         for line in lines:
253             texts = {w["text"] for w in line}
254             if "Narration" in texts and any(t.startswith("Closing") for t in texts):
255                 header_top = _line_top(line)
256                 break
257         if header_top is None:
258             return []
259         return [line for line in lines if _line_top(line) > header_top + 1]
260
261
262     def _extract_account_number(text: str) -> str | None:
263         """Account number via the HDFC profile (bank knowledge as data; same pattern as
264         before)."""
265         return HDFC_PROFILE.first_account(text)
266
267
268     def parse_statement(pdf_path: Path | str, password: str = "") -> Statement:
269         """Parse an HDFC statement PDF into a :class:`Statement` of transactions.
270
271         ``password`` is forwarded to pdfplumber for encrypted statements and used
272         in-memory only ? never stored or logged.
273         """
274         path = Path(pdf_path)
275         transactions: list[Transaction] = []
276         account_number: str | None = None
277         with pdfplumber.open(str(path), password=password or "") as pdf:
278             for page_no, page in enumerate(pdf.pages):
279                 if account_number is None and page_no < 2:
280                     account_number = _extract_account_number(page.extract_text() or "")
281                 words = page.extract_words(use_text_flow=False, keep_blank_chars=False)
282                 body = _table_body(_group_lines(words))
283                 transactions.extend(extract_transactions_from_lines(body, page_no))
284         return Statement(path=path, transactions=transactions,
285         account_number=account_number)
286
287
288     def reconcile(statement: Statement, tol: float = 0.01) -> ReconcileReport:
289         """Check the running balance via the shared reconciler (closing[i] ==
290         closing[i-1] - withdrawal + deposit).
291
292         Spans page breaks (transactions are in document order), so a clean report also
293         validates page stitching. Reports counts only ? no amounts.
294         """
295         entries = [(t.withdrawal, t.deposit, t.closing_balance) for t in
296         statement.transactions]
297         return reconcile_running_balance(entries, tol)
298
299
300     def _parse_summary_lines(lines: list[str]) -> StatementSummary | None:
301         """Pure core: read HDFC's SUMMARY block from extracted text lines.
302
303         Layout (stable across statements)::
304
305             SUMMARY
306             Opening Balance  Debit Amount  Credit Amount  Closing Balance
307             <opening>        <debitAmt>    <creditAmt>    <closing>
308             Debit Count  Credit Count
309             <debitCount> <creditCount>

```

```

308     The values land on their own lines, so we take the first post-SUMMARY line
309     carrying ?4 money tokens (the balances) and the first carrying ?2 bare
310     integers and no money (the counts).
311     """
312     start = next((i for i, ln in enumerate(lines) if ln.strip() == "SUMMARY"), None)
313     if start is None:
314         return None
315     balances: list[float] | None = None
316     counts: list[int] | None = None
317     for ln in lines[start + 1 : start + 8]:
318         tokens = ln.split()
319         monies = [v for v in (_to_amount(t) for t in tokens) if v is not None]
320         ints = [int(t) for t in tokens if t.isdigit()]
321         if balances is None and len(monies) >= 4:
322             balances = monies[:4]
323         elif counts is None and not monies and len(ints) >= 2:
324             counts = ints[:2]
325     if balances is None and counts is None:
326         return None
327     bvals: list[float | None] = [*balances] if balances else [None, None, None, None]
328     opening, debit_amt, credit_amt, closing = bvals[:4]
329     cvals: list[int | None] = [*counts] if counts else [None, None]
330     debit_count, credit_count = cvals[:2]
331     return StatementSummary(
332         opening_balance=opening,
333         closing_balance=closing,
334         debit_count=debit_count,
335         credit_count=credit_count,
336         debit_amount=debit_amt,
337         credit_amount=credit_amt,
338     )
339
340
341 def parse_summary(pdf_path: Path | str, password: str = "") -> StatementSummary | None:
342     """Read the bank's own SUMMARY figures from the statement, or None if absent."""
343     with pdfplumber.open(str(Path(pdf_path)), password=password or "") as pdf:
344         for page in pdf.pages:
345             text = page.extract_text() or ""
346             if "SUMMARY" in text:
347                 summary = _parse_summary_lines(text.splitlines())
348                 if summary is not None:
349                     return summary
350     return None
351
352
353 def validate_against_summary(
354     statement: Statement, summary: StatementSummary, tol: float = 0.01
355 ) -> ValidationReport:
356     """Cross-check a parse against the bank's SUMMARY: reconciliation + bookends.
357
358     Maps HDFC's rows onto the canonical concepts and delegates to the shared verifier
359     (:func:`muneem.parsers.validation.verify`): reconciliation catches any mid-statement
360     error/omission, the Dr/Cr counts catch any extra/missing row, and the opening &
361     closing
362     balances catch a missing first/last row.
363     """
364     rows = [t.to_concept() for t in statement.transactions]
365     return verify(rows, summary, tol)
366
367
368 def write_csv(statement: Statement, out_path: Path | str) -> Path:
369     """Write parsed transactions to CSV for local QA (the 'export to a sheet' idea).
370
371     Writes real data ? call only for a local, gitignored destination.
372     """

```

```

372 out = Path(out_path)
373 out.parent.mkdir(parents=True, exist_ok=True)
374 with open(out, "w", newline="", encoding="utf-8") as fh:
375     writer = csv.writer(fh)
376     writer.writerow(
377         [
378             "date",
379             "value_date",
380             "narration",
381             "ref",
382             "withdrawal",
383             "deposit",
384             "closing_balance",
385             "page",
386         ]
387     )
388     for t in statement.transactions:
389         writer.writerow(
390             [
391                 t.date,
392                 t.value_date or "",
393                 t.narration,
394                 t.ref or "",
395                 f"{t.withdrawal:.2f}",
396                 f"{t.deposit:.2f}",
397                 f"{t.closing_balance:.2f}",
398                 t.page,
399             ]
400         )
401     return out
402
403
404 @register_parser
405 class HDFCParser(StatementParser):
406     """Registered parser for HDFC savings / transaction-account statements."""
407
408     def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
409         low = (text or "").lower()
410         is_hdfc = "hdfc bank" in low or "hdfcbank" in low or "hdfcbank" in (sender or
"".lower())
411         is_statement = "savings account details" in low or (
412             "narration" in low and "closing balance" in low
413         )
414         return is_hdfc and is_statement
415
416     def parse_and_insert(
417         self,
418         pdf_path: Path,
419         conn,
420         doc_id: int,
421         password: str | None = None,
422     ) -> int:
423         """Parse, insert rows into ``hdfc_transactions``, and record parse quality.
424
425         Sets ``documents.status`` to 'reconciled' / 'unreconciled' based on the
426         balance walk (and the bank's SUMMARY when present) so a downstream consumer
427         can tell a trustworthy parse from a suspect one. Returns the row count.
428         """
429         try:
430             stmt = parse_statement(pdf_path, password=password or "")
431         except ParseError:
432             raise
433         except Exception as exc:
434             raise ParseError(f"failed to parse HDFC statement

```



```

{redact_filename(pdf_path)}}") from exc
435         for t in stmt.transactions:
436             dbmod.insert_hdfc_transaction(
437                 conn,
438                 document_id=doc_id,
439                 account_number=stmt.account_number,
440                 txn_date=t.date,
441                 value_date=t.value_date,
442                 particulars=t.narration,
443                 ref=t.ref,
444                 withdrawal=t.withdrawal,
445                 deposit=t.deposit,
446                 closing_balance=t.closing_balance,
447             )
448         summary = parse_summary(pdf_path, password=password or "")
449         ok = (
450             validate_against_summary(stmt, summary).ok
451             if summary is not None
452             else reconcile(stmt).ok
453         )
454         verdict = Confidence.RECONCILED if ok else Confidence.UNRECONCILED
455         dbmod.record_parse(
456             conn, doc_id, confidence=verdict.value, parser="HDFCParser",
457             parser_version=__version__
458         )
459         return len(stmt.transactions)

```

3. user (2026-06-10T16:28:28.905Z)

```

400
401     def list_offers(conn: sqlite3.Connection) -> list[sqlite3.Row]:
402         return conn.execute(
403             """
404             SELECT o.id, o.merchant, o.promo_code, o.expiry_raw, o.description, o.source,
405                    d.external_id, d.path
406             FROM offers o
407             JOIN documents d ON d.id = o.document_id
408             ORDER BY o.id
409             """
410         ).fetchall()
411
412
413     def insert_axis_transaction(conn: sqlite3.Connection, **fields: object) -> int:
414         return _insert_row(conn, "axis_transactions", fields)
415
416
417     def insert_axis_cc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
418         return _insert_row(conn, "axis_cc_transactions", fields)
419
420
421     def list_axis_cc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
422     list[sqlite3.Row]:
423         return conn.execute(
424             "SELECT * FROM axis_cc_transactions ORDER BY id DESC LIMIT ?", (limit,)
425         ).fetchall()
426
427     def list_axis_transactions(conn: sqlite3.Connection, limit: int = 50) ->
428     list[sqlite3.Row]:
429         return conn.execute(
430             "SELECT * FROM axis_transactions ORDER BY id DESC LIMIT ?", (limit,)
431         ).fetchall()
432

```

```

433     def insert_hdfc_transaction(conn: sqlite3.Connection, **fields: object) -> int:
434         return _insert_row(conn, "hdfc_transactions", fields)
435
436
437     def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
438         return conn.execute(
439             "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?", (limit,)
440         ).fetchall()
441
442
443     def insert_au_transaction(conn: sqlite3.Connection, **fields: object) -> int:
444         return _insert_row(conn, "au_transactions", fields)
445
446
447     def list_au_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
448         return conn.execute(
449             "SELECT * FROM au_transactions ORDER BY id DESC LIMIT ?", (limit,)
450         ).fetchall()
451

```

4. user (2026-06-10T16:28:35.851Z)

```

1     """Axis Bank statement parsers: savings/transaction account + credit card.
2
3     **Savings runs on the shared engine-driven path** (:mod:`muneem.parsers.savings`): per-
table +
4     coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
gated** so a
5     vintage it can't read stores nothing (`unreconciled`) rather than mislabeled rows.
Axis specifics
6     live entirely in `AXIS_PROFILE`; this replaced the old fixed-column `extract_tables`
mapping.
7
8     Credit-card statements carry no running balance, so `AxisCreditCardParser` can't
reconcile
9     (parse/persist split + count only). No statement content is logged here.
10    """
11
12    from __future__ import annotations
13
14    import re
15    import sqlite3
16    from dataclasses import dataclass
17    from pathlib import Path
18
19    import pdfplumber
20
21    from muneem import __version__
22    from muneem import db as dbmod
23    from muneem.errors import ParseError
24    from muneem.redaction import redact_filename
25
26    from . import register_parser
27    from .base import StatementParser
28    from .profiles import AXIS_PROFILE
29    from .savings import SavingsParse, parse_savings, parse_statement_summary,
txn_line_predicate
30    from .validation import Confidence, parse_amount
31
32    _CC_DATE_RE = re.compile(r"\d{2}/\d{2}/\d{4}") # Axis credit card: dd/mm/yyyy
33    _CARD_RES = [re.compile(r"(\d{4}XXXX\d{4})"), re.compile(r"(\d{6}\*\d{4})")]
34
35

```

```

36     def _first_match(text: str, patterns: list[re.Pattern]) -> str | None:
37         for rx in patterns:
38             m = rx.search(text)
39             if m:
40                 return m.group(1).strip()
41         return None
42
43
44     # --- Savings / transaction account (concept core)
-----
45
46
47     # Back-compat names; the savings logic now lives in muneem.parsers.savings (shared with
AU Bank).
48     AxisSavings = SavingsParse
49     parse_axis_summary = parse_statement_summary
50     _is_txn_line = txn_line_predicate(Axis_PROFILE)
51
52
53     def parse_axis_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
54         """Parse an Axis savings statement via the shared engine-driven path
(savings.parse_savings)."""
55         return parse_savings(pdf_path, Axis_PROFILE, password)
56
57
58     @register_parser
59     class AxisParser(StatementParser):
60         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
61             if sender and "axisbank.com" in sender.lower():
62                 return True
63             t = text or ""
64             return "Axis Bank" in t and ("Statement of Account" in t or "Account Statement"
in t)
65
66     def parse_and_insert(
67         self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
68     ) -> int:
69         """Persist a savings parse **only if it reconciles**; otherwise store nothing.
70
71         Sets ``documents.status`` to ``reconciled`` / ``unreconciled`` accordingly, so a
layout the
72         parser can't yet read surfaces loudly instead of saving mislabeled rows.
73         """
74         try:
75             parsed = parse_axis_savings(pdf_path, password)
76         except ParseError:
77             raise
78         except Exception as exc:
79             # Filename only ? never statement text; original cause kept via `from`.
80             name = redact_filename(pdf_path)
81             raise ParseError(f"failed to parse Axis statement {name}") from exc
82
83         if not parsed.verified:
84             dbmod.record_parse(
85                 conn,
86                 doc_id,
87                 confidence=Confidence.UNRECONCILED.value,
88                 parser="AxisParser",
89                 parser_version=__version__,
90             )
91             return 0
92
93         for r in parsed.rows:

```

```

94         dbmod.insert_axis_transaction(
95             conn,
96             document_id=doc_id,
97             account_number=parsed.account_number,
98             txn_date=r.txn_date,
99             value_date=r.value_date or "",
100            particulars=r.narration,
101            debit=r.debit,
102            credit=r.credit,
103            balance=r.balance,
104        )
105        dbmod.record_parse(
106            conn,
107            doc_id,
108            confidence=Confidence.RECONCILED.value,
109            parser="AxisParser",
110            parser_version=__version__,
111        )
112        return len(parsed.rows)
113
114
115    # --- Credit card (no running balance; count only)
116    -----
117
118    @dataclass
119    class AxisCCTxn:
120        txn_date: str
121        particulars: str
122        merchant_category: str
123        amount: float
124        type: str # "Cr" or "Dr"
125
126
127    def _rows_from_cc_tables(tables: list) -> list[AxisCCTxn]:
128        """Build credit-card transactions from extracted tables. Pure: no PDF, no DB."""
129        rows: list[AxisCCTxn] = []
130        for table in tables or []:
131            for row in table or []:
132                if not row or len(row) < 9:
133                    continue
134                date_str = str(row[0]).strip() if row[0] else ""
135                if not _CC_DATE_RE.match(date_str):
136                    continue
137                amount_str = str(row[8]).strip() if row[8] else ""
138                is_cr = "Cr" in amount_str
139                amount = parse_amount(re.sub(r"^\d.", "", amount_str)) or 0.0
140                rows.append(
141                    AxisCCTxn(
142                        txn_date=date_str,
143                        particulars=str(row[2]).strip() if row[2] else "",
144                        merchant_category=str(row[7]).strip() if row[7] else "",
145                        amount=amount,
146                        type="Cr" if is_cr else "Dr",
147                    )
148                )
149        return rows
150
151
152    def parse_axis_cc(pdf_path: Path, password: str | None = None) -> tuple[str,
list[AxisCCTxn]]:
153        """Open an Axis credit-card statement and return (card_number, transactions)."""
154        card_number = "UNKNOWN"
155        rows: list[AxisCCTxn] = []
156        with pdfplumber.open(pdf_path, password=password) as pdf:

```

```

157         head = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(3,
len(pdf.pages))))
158         card_number = _first_match(head, _CARD_RES) or "UNKNOWN"
159         for page in pdf.pages:
160             rows.extend(_rows_from_cc_tables(page.extract_tables()))
161         return card_number, rows
162
163
164     @register_parser
165     class AxisCreditCardParser(StatementParser):
166         def can_handle(self, text: str, sender: str | None = None, subject: str | None =
None) -> bool:
167             return "Axis Bank" in text and "Credit Card Statement" in text
168
169         def parse_and_insert(
170             self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
None = None
171         ) -> int:
172             try:
173                 card_number, rows = parse_axis_cc(pdf_path, password)
174             except ParseError:
175                 raise
176             except Exception as exc:
177                 raise ParseError(
178                     f"failed to parse Axis CC statement {redact_filename(pdf_path)}"
179                 ) from exc
180             for t in rows:
181                 dbmod.insert_axis_cc_transaction(
182                     conn,
183                     document_id=doc_id,
184                     account_number=card_number,
185                     txn_date=t.txn_date,
186                     particulars=t.particulars,
187                     merchant_category=t.merchant_category,
188                     amount=t.amount,
189                     type=t.type,
190                 )
191             # No running balance to reconcile; documents.status stays as the caller set it.
192
193             # record provenance (with no confidence verdict).
194             dbmod.record_provenance(
195                 conn,
196                 document_id=doc_id,
197                 source_type="pdf_statement",
198                 parser="AxisCreditCardParser",
199                 parser_version=__version__,
200                 mapping_layer="deterministic",
201             )
202             return len(rows)

```

5. user (2026-06-10T16:28:36.389Z)

```

320         return
321     for row in rows:
322         who = row["merchant"] or row["external_id"] or "?"
323         when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
324         tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
325         typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")
326
327
328     @txns_app.command("list")
329     def txns_list(
330         bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc,
axis)."),

```

```

331         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
332         limit: int = typer.Option(50, help="Max number of transactions to show."),
333     ) -> None:
334         """List parsed transactions for a specific bank."""
335         from muneem import db as dbmod
336
337         db_path = db or load_config().db_path
338         if not Path(db_path).exists():
339             typer.echo("database not created yet.")
340             return
341
342         bank = bank.lower()
343         if bank not in BANK_DISPATCH:
344             typer.echo(f"Unknown bank: {bank}", err=True)
345             return
346
347         list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
348         with _ledger(db_path) as conn:
349             dbmod.init_db(conn)
350             rows = list_fn(conn, limit=limit)
351
352         if not rows:
353             typer.echo(f"no {bank} transactions found.")
354             return
355         for row in rows:
356             if bank == "axis_cc":
357                 amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
358             else:
359                 amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
360             part = (row["particulars"] or "")[:50]
361             typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} |
{amount:>10}")
362
363
364     @db_app.command("status")
365     def db_status(
366         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),
367     ) -> None:
368         """Show the ledger's encryption state (no statement data is read or printed)."""
369         from muneem import db as dbmod
370         from muneem.db_key import peek_key
371
372         db_path = db or load_config().db_path
373         key_state = "present in keychain" if peek_key() else "not yet minted"
374         if not Path(db_path).exists():
375             typer.echo(f"ledger : {db_path} (does not exist yet)")
376             typer.echo(f"key      : {key_state}")
377             return
378         if dbmod.is_plaintext_sqlite(db_path):
379             state = "PLAINTEXT ? run 'muneem db encrypt' to harden it"
380         else:
381             state = "encrypted (SQLCipher)"
382         typer.echo(f"ledger : {db_path}")
383         typer.echo(f"state  : {state}")
384         typer.echo(f"key    : {key_state}")
385
386
387     @db_app.command("encrypt")
388     def db_encrypt(
389         db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data
dir)."),

```

The core claims are confirmed so far. Let me check the AU parser, `record_parse`, and tests to see if any test intentionally codifies the HDFC behavior.

7. user (2026-06-10T16:29:04.258Z)

```

1  """AU Small Finance Bank savings / transaction-account statement parser.
2
3  Runs on the **shared engine-driven path** (:mod:`muneem.parsers.savings`): per-table +
4  coordinate-clustered candidates ? engine mapping ? reconciliation oracle, **oracle-
5  gated** so a
6  vintage it can't read stores nothing rather than mislabeled rows. The AU specifics are
7  entirely in
8  ``AUBANK_PROFILE`` ? this parser is mostly the DB wiring. No statement content is logged
9  here.
10
11  """
12
13  from __future__ import annotations
14
15  import sqlite3
16  from pathlib import Path
17
18  from muneem import __version__
19  from muneem import db as dbmod
20  from muneem.errors import ParseError
21  from muneem.redaction import redact_filename
22
23  from . import register_parser
24  from .base import StatementParser
25  from .profiles import AUBANK_PROFILE
26  from .savings import SavingsParse, parse_savings
27  from .validation import Confidence
28
29
30  def parse_au_savings(pdf_path: Path, password: str | None = None) -> SavingsParse:
31  """Parse an AU Bank savings statement via the shared engine-driven path."""
32  return parse_savings(pdf_path, AUBANK_PROFILE, password)
33
34
35  @register_parser
36  class AUParser(StatementParser):
37  def can_handle(self, text: str, sender: str | None = None, subject: str | None =
38  None) -> bool:
39  if sender and "aubank" in sender.lower():
40  return True
41  low = (text or "").lower()
42  return "au small finance bank" in low or "aubank" in low
43
44  def parse_and_insert(
45  self, pdf_path: Path, conn: sqlite3.Connection, doc_id: int, password: str |
46  None = None
47  ) -> int:
48  """Persist a savings parse **only if it reconciles**; otherwise store
49  nothing."""
50  try:
51  parsed = parse_au_savings(pdf_path, password)
52  except ParseError:
53  raise
54  except Exception as exc:
55  raise ParseError(f"failed to parse AU statement
56  {redact_filename(pdf_path)}") from exc
57
58  if not parsed.verified:
59  dbmod.record_parse(
60  conn,
61  doc id,

```

```

54         confidence=Confidence.UNRECONCILED.value,
55         parser="AUParser",
56         parser_version=__version__,
57     )
58     return 0
59
60     for r in parsed.rows:
61         dbmod.insert_au_transaction(
62             conn,
63             document_id=doc_id,
64             account_number=parsed.account_number,
65             txn_date=r.txn_date,
66             value_date=r.value_date or "",
67             particulars=r.narration,
68             debit=r.debit,
69             credit=r.credit,
70             balance=r.balance,
71         )
72         dbmod.record_parse(
73             conn,
74             doc_id,
75             confidence=Confidence.RECONCILED.value,
76             parser="AUParser",
77             parser_version=__version__,
78         )
79     return len(parsed.rows)
80

```

8. user (2026-06-10T16:29:05.539Z)

```

364-         "parser_version=excluded.parser_version, mapping_layer=excluded.mapping_layer, "
365-         "confidence=excluded.confidence",
366-         (document_id, source_type, parser, parser_version, mapping_layer, confidence),
367-     )
368-
369-
370: def record_parse(
371-     conn: sqlite3.Connection,
372-     document_id: int,
373-     *,
374-     confidence: str,
375-     parser: str,
376-     source_type: str = "pdf_statement",
377-     parser_version: str | None = None,
378-     mapping_layer: str = "deterministic",
379-) -> None:
380-     """Record a parse outcome: set ``documents.status`` and upsert provenance (one per
381-     parse)."""
382-     conn.execute("UPDATE documents SET status = ? WHERE id = ?", (confidence, document_id))
383-     record_provenance(
384-         conn,
385-         document_id=document_id,
386-         source_type=source_type,
387-         parser=parser,
388-         parser_version=parser_version,
389-     )
390-

```

9. user (2026-06-10T16:29:15.545Z)

```

C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-71-         == 1
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-72-     )
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-73-
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-74- tests\test_provenance.py:75: def
test_unreconciled_parse_records_unreconciled_provenance(monkeypatch):
tests\test_provenance.py-76-     monkeypatch.setattr(au, "parse_au_savings", lambda *a, **k:

```



```

SavingsParse("AU1", [], False))
tests\test_provenance.py-77-         with dbmod.connect(":memory:") as conn:
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-78-             dbmod.init_db(conn)
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-79-             doc = _doc(conn)
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-80-             n =
au.AUParser().parse_and_insert(Path("x.pdf"), conn, password=None)
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-81-             assert n == 0
tests\test_provenance.py:82:         assert dbmod.get_provenance(conn, doc)["confidence"] ==
"unreconciled"
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-83-
C:\Users\avidu\Projects\muneeem\tests\test_provenance.py-84-
tests\test_provenance.py-85-def
test_credit_card_parser_records_provenance_without_confidence(monkeypatch):
tests\test_provenance.py-86-     monkeypatch.setattr(axis, "parse_axis_cc", lambda *a, **k:
("CARD1234", []))
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-72-         assert r.verified and
len(r.rows) == 2
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-73-         assert r.rows[0].debit == 500.0
and r.rows[0].balance is None
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-74-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-75-
tests\parsers\test_axis.py:76:def test_savings_refuses_when_unreconciled():
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-77-     table = [
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-78-         ["Tran Date",
"Transaction", "Withdrawals", "Deposits", "Balance"],
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-79-         ["01-02-2026", "UPI",
"500.00", "", "9500.00"],
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-80-         ["02-02-2026", "Salary",
"", "2000.00", "1.00"], # balance breaks the walk
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-110-         doc_id =
dbmod.insert_document(conn, source="file", sha256="ax2", status="parsed")
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-111-         n =
AxisParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-112-         assert n == 0
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-113-         assert
dbmod.list_axis_transactions(conn) == []
tests\parsers\test_axis.py:114:         assert dbmod.get_document_by_sha(conn, "ax2")["status"]
== "unreconciled"
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-115-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-116-
tests\parsers\test_axis.py-117-def test_rows_from_cc_tables():
C:\Users\avidu\Projects\muneeem\tests\parsers\test_axis.py-118-     tables = [
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_validation.py-90-     r =
verify(_concept_rows(), StatementConcepts())
C:\Users\avidu\Projects\muneeem\tests\parsers\test_validation.py-91-     assert r.reconciles
C:\Users\avidu\Projects\muneeem\tests\parsers\test_validation.py-92-     assert r.debit_count_ok
is None and r.credit_count_ok is None
C:\Users\avidu\Projects\muneeem\tests\parsers\test_validation.py-93-     assert r.opening_ok is
None and r.closing_ok is None
tests\parsers\test_validation.py:94:     assert not r.ok and r.confidence is
Confidence.UNRECONCILED
C:\Users\avidu\Projects\muneeem\tests\parsers\test_validation.py-95-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_validation.py-96-
tests\parsers\test_validation.py-97-def test_verify_catches_wrong_count():
C:\Users\avidu\Projects\muneeem\tests\parsers\test_validation.py-98-     concepts =
StatementConcepts(
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_au.py-67-         doc_id =
dbmod.insert_document(conn, source="file", sha256="au2", status="parsed")
C:\Users\avidu\Projects\muneeem\tests\parsers\test_au.py-68-         n =
AUParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
C:\Users\avidu\Projects\muneeem\tests\parsers\test_au.py-69-         assert n == 0

```

```

C:\Users\avidu\Projects\muneeem\tests\parsers\test_au.py-70-          assert
dbmod.list_au_transactions(conn) == []
tests\parsers\test_au.py:71:          assert dbmod.get_document_by_sha(conn, "au2")["status"] ==
"unreconciled"
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-20-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-21-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-22-@pytest.mark.local_statements
tests\parsers\test_local_au.py-23-@pytest.mark.parametrize("pdf_path", AU_PDFS, ids=lambda p:
p.name)
tests\parsers\test_local_au.py:24:def test_local_au_never_stores_unreconciled(pdf_path,
tmp_path):
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-25-    pw = (
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-26-    get_password_for_bank("aubank")
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-27-    or
get_password_for_bank("au")
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-28-    or
os.environ.get("MUNEEM_TEST_AU_PW")
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-43-    status =
dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-44-    stored =
dbmod.list_au_transactions(conn)
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-45-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-46-    assert (n == 0 and stored
== []) or status == "reconciled", (
tests\parsers\test_local_au.py:47:        f"{pdf_path.name}: stored {n} row(s) with
status={status} ? must not persist unreconciled"
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_au.py-48-    )
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-22-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-23-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-24-@pytest.mark.local_statements
tests\parsers\test_local_axis.py-25-@pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda
p: p.name)
tests\parsers\test_local_axis.py:26:def test_local_axis_never_stores_unreconciled(pdf_path,
tmp_path):
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-27-    pw =
get_password_for_bank("axis") or os.environ.get("MUNEEM_TEST_AXIS_PW")
tests\parsers\test_local_axis.py-28-    if not pw:
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-29-    pytest.skip("needs
the Axis password (password_rules.yaml or MUNEEM_TEST_AXIS_PW)")
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-30-
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-41-    status =
dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-42-    stored =
dbmod.list_axis_transactions(conn)
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-43-
tests\parsers\test_local_axis.py-44-    # Oracle-gated invariant: the savings parser either
stores nothing (and flags the
tests\parsers\test_local_axis.py:45-    # document unreconciled) or, if it stored rows, they
reconciled. Never garbage.
tests\parsers\test_local_axis.py-46-    if type(parser).__name__ == "AxisParser":
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-47-    assert (n == 0 and
stored == []) or status == "reconciled", (
tests\parsers\test_local_axis.py-48-        f"{pdf_path.name}: stored {n} row(s) with
status={status} ? "
tests\parsers\test_local_axis.py:49-        "stop-gap must not persist unreconciled rows"
C:\Users\avidu\Projects\muneeem\tests\parsers\test_local_axis.py-50-    )
--
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-193-    assert {r["particulars"]}
for r in rows} # narration mapped to particulars
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-194-    assert

```

```

dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-195-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-196-
tests\parsers\test_hdfc.py:197: def test_parse_and_insert_records_unreconciled(tmp_path,
monkeypatch):
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-198-         broken = _parse_sample()
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-199-         broken[2].closing_balance =
1.0 # break the balance walk
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-200-         monkeypatch.setattr(
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-201-             hdfc,
--
tests\parsers\test_hdfc.py-206-         with dbmod.connect(tmp_path \ "t.db") as conn:
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-207-             dbmod.init_db(conn)
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-208-             doc_id =
dbmod.insert_document(conn, source="file", sha256="h2", status="parsed")
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-209-
hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
tests\parsers\test_hdfc.py:210:         assert dbmod.get_document_by_sha(conn, "h2")["status"] ==
"unreconciled"
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-211-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-212-
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-213-@pytest.mark.local_statements
C:\Users\avidu\Projects\muneeem\tests\parsers\test_hdfc.py-214-@pytest.mark.parametrize(

```

10. user (2026-06-10T16:29:16.225Z)

```

src\muneeem\cli.py:56:     "hdfc": (dbmod.list_hdfc_transactions, "withdrawal", "deposit"),
src\muneeem\db.py:55:-- We use separate tables for each bank (e.g. axis_transactions,
hdfc_transactions)
src\muneeem\db.py:94:CREATE TABLE IF NOT EXISTS hdfc_transactions (
src\muneeem\db.py:434:     return _insert_row(conn, "hdfc_transactions", fields)
src\muneeem\db.py:437: def list_hdfc_transactions(conn: sqlite3.Connection, limit: int = 50) ->
list[sqlite3.Row]:
src\muneeem\db.py:439:     "SELECT * FROM hdfc_transactions ORDER BY id DESC LIMIT ?",
(limit,)
src\muneeem\parsers\hdfc.py:423:     """Parse, insert rows into ``hdfc_transactions``, and
record parse quality.

```

11. user (2026-06-10T16:30:03.968Z)

```

132:         status="parsed",
136:         status = doc["status"] if doc else "parsed"
140:         f"via {parser.__class__.__name__} [{status}]"
147:     Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document status
177:         conn, source="file", external_id=pdf.name, sha256=ex.sha256, status="parsed"
181:         status = doc["status"] if doc else "parsed"
187:     return (status, count, name)
209:     an in-memory DB; only counts / statuses / parser names print ? never statement values.
285:         status="parsed",
364:@db_app.command("status")
365: def db_status(

```

12. user (2026-06-10T16:30:05.192Z)

```

160         assert c.txn_date == "05/02/2026" and c.narration == "UPI-SWIGGY"
161         assert c.debit == 500.0 and c.credit == 0.0 and c.balance == 9500.0
162         assert c.value_date == "04/02/2026" and c.reference == "123456789012"
163
164
165     def test_can_handle():
166         parser = hdfc.HDFCParser()
167         statement = "HDFC BANK Savings Account Details Narration Closing Balance"
168         assert parser.can_handle(statement)
169         assert parser.can_handle(statement, sender="alerts@hdfcbank.net")

```

```

170         # sender alone, with no statement markers, must NOT match (e.g. a promo from HDFC)
171         assert not parser.can_handle("anything", sender="alerts@hdfcbank.net")
172         assert not parser.can_handle("HDFC Bank promo: use code SAVE50")
173         assert not parser.can_handle("Axis Bank Statement of Account Narration Closing
Balance")
174
175
176     def test_parse_and_insert_records_reconciled(tmp_path, monkeypatch):
177         monkeypatch.setattr(hdfc, "parse_statement", lambda *a, **k: _sample_statement())
178         monkeypatch.setattr(
179             hdfc,
180             "parse_summary",
181             lambda *a, **k: hdfc.StatementSummary(
182                 opening_balance=10000.0, closing_balance=9999.50, debit_count=2,
credit_count=1
183             ),
184         )
185         with dbmod.connect(tmp_path / "t.db") as conn:
186             dbmod.init_db(conn)
187             doc_id = dbmod.insert_document(conn, source="file", sha256="h1",
status="parsed")
188             n = hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
189             assert n == 3
190             rows = dbmod.list_hdfc_transactions(conn, limit=100)
191             assert len(rows) == 3
192             assert rows[0]["account_number"] == "501000000000000"
193             assert {r["particulars"] for r in rows} # narration mapped to particulars
194             assert dbmod.get_document_by_sha(conn, "h1")["status"] == "reconciled"
195
196
197     def test_parse_and_insert_records_unreconciled(tmp_path, monkeypatch):
198         broken = _parse_sample()
199         broken[2].closing_balance = 1.0 # break the balance walk
200         monkeypatch.setattr(
201             hdfc,
202             "parse_statement",
203             lambda *a, **k: hdfc.Statement(path=Path("x"), transactions=broken),
204         )
205         monkeypatch.setattr(hdfc, "parse_summary", lambda *a, **k: None) # fall back to
reconcile()
206         with dbmod.connect(tmp_path / "t.db") as conn:
207             dbmod.init_db(conn)
208             doc_id = dbmod.insert_document(conn, source="file", sha256="h2",
status="parsed")
209             hdfc.HDFCParser().parse_and_insert(Path("x.pdf"), conn, doc_id)
210             assert dbmod.get_document_by_sha(conn, "h2")["status"] == "unreconciled"
211
212
213     @pytest.mark.local_statements
214     @pytest.mark.parametrize(
215         "pdf_path",
216         sorted(Path("testdata/hdfc-statements").glob("*.pdf")),
217         ids=lambda p: p.name,
218     )
219     def test_real_hdfc_statement_validates(pdf_path, tmp_path):

```

13. user (2026-06-10T16:30:40.862Z)

```

125         doc_id = dbmod.insert_document(
126             conn,
127             source="file",
128             external_id=pdf.name,
129             doc_type=doc_type,
130             sha256=ex.sha256,
131             path=str(pdf.resolve()),

```

```

132         status="parsed",
133     )
134     count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
135     doc = dbmod.get_document_by_sha(conn, ex.sha256)
136     status = doc["status"] if doc else "parsed"
137
138     typer.echo(
139         f"ingested {pdf.name} (doc {doc_id}); parsed {count} transaction(s) "
140         f"via {parser.__class__.__name__} [{status}]"
141     )
142
143
144     def _eval_statement(pdf: Path, *, interactive: bool) -> tuple[str, int, str | None]:
145         """Run one statement through the full parse pipeline into an in-memory DB.
146
147         Returns ``(outcome, row_count, parser_name)`` where ``outcome`` is the document
148         status
149         (`reconciled` / `unreconciled`) or a failure tag (`locked` / `no-parser` /
150         `error`).
151         Nothing is persisted and no statement value is surfaced ? only the verdict and
152         counts.
153         """
154         from muneem import db as dbmod
155         from muneem.extract import extract
156         from muneem.parsers import get_parser
157         from muneem.statement_unlock import resolve_password
158         from muneem.unlock import is_encrypted
159
160         password = ""
161         if is_encrypted(pdf):
162             password = resolve_password(pdf, interactive=interactive) or ""
163             if not password:
164                 return ("locked", 0, None)
165         try:
166             ex = extract(pdf, password=password)
167             parser = get_parser(ex.text)
168         except Exception as exc: # eval must not crash on one bad file; record the redacted
169             reason
170             _log.warning(
171                 "eval: extract/route failed for %s (%s)", redact_filename(pdf),
172                 type(exc).__name__
173             )
174             return ("error", 0, None)
175         if parser is None:
176             return ("no-parser", 0, None)
177         name = parser.__class__.__name__
178         try:
179             with dbmod.connect(":memory:") as conn:
180                 dbmod.init_db(conn)
181                 doc_id = dbmod.insert_document(
182                     conn, source="file", external_id=pdf.name, sha256=ex.sha256,
183                     status="parsed"
184                 )
185                 count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
186                 doc = dbmod.get_document_by_sha(conn, ex.sha256)
187                 status = doc["status"] if doc else "parsed"
188             except Exception as exc:
189                 _log.warning(
190                     "eval: parse/store failed for %s (%s)", redact_filename(pdf),
191                     type(exc).__name__
192                 )
193                 return ("error", 0, name)
194         return (status, count, name)

```

14. user (2026-06-10T16:31:06.794Z)

Structured output provided successfully

15. assistant (2026-06-10T16:31:14.092Z)

Verification complete. The finding is confirmed as real with high severity justified, with one sub-claim (the fabricated 0.0 closing balance) refuted as effectively unreachable.